Session 17 — Study Guide: Tables, Keys, and a Second Kind of Database

Where the session ends

A real, six-table PostgreSQL schema exists in a new, dedicated practice project (`sql-practice/`) — `users`, `workspaces`, `workspace_members`, `projects`, `issues`, `comments` — modelling B4F Board's own real domain a second way. Every real constraint (PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK) is proven, and every real foreign key's own delete behavior is deliberately chosen and proven. `api/` (the real Nest+Mongo backend) is completely untouched — this is parallel practice, not a migration.

The real starting point: a short, bounded verification, not a re-teach

Session 16's own homework (ownership-based Issue authorization) was assigned at the real end of that session, less than two real teaching days before this one. It is confirmed absent from every real student branch — not a surprise, since almost no time has elapsed. This session opens with a short, bounded check (not a re-teach): students who finished it demonstrate the real proof; students who didn't get real working time now. Block 3 (Authentication/Authorization) is then explicitly closed before new material begins.

Why a second kind of database, and why now

Every real domain fact this program has used since Session 15 — a Workspace, its real `WorkspaceMember` rows, its real Projects — already lived in MongoDB. That was never framed as a mistake, and it isn't one here either: `projects/b4f-board/specification/DATABASE-PROGRESSION.md` is explicit that both database stages model the SAME real domain, on purpose, because the contrast itself is a core lesson. Today's work is not a detour — `WorkspaceMember` was already modelled as its own referenced collection in Mongo specifically so this exact relational migration would be smooth later.

A table is a stricter, more declarative version of something you already built

`CREATE TABLE users (id SERIAL PRIMARY KEY, email TEXT UNIQUE, ...)` fixes a real, named, typed set of columns every row shares. `SERIAL PRIMARY KEY` is the structural version of the hand-rolled auto-increment `id` pattern every Mongo service in this app already uses. `UNIQUE` is the structural version of a Mongo index, or an application-level uniqueness check — except the database itself now refuses the duplicate, with a real, specific error.

FOREIGN KEY — a real, enforced link, not a trusted number

`owner_id INTEGER NOT NULL REFERENCES users(id)` means Postgres itself refuses to store a `workspaces` row whose `owner_id` names a user that does not exist. In the real Mongo-backed app today, nothing stops a bug from writing a document whose `ownerId` points at nothing — it would simply sit there,

silently wrong. Here, it cannot happen at all.

N:N with an attribute needs its own table

A real user can belong to many workspaces; a real workspace can have many users. That's a many-to-many relationship, and it cannot be expressed with one foreign key on either side. It needs its own table (`workspace_members`), referencing both, carrying its own extra attribute (`role`). `UNIQUE (workspace_id, user_id)` is the exact same real rule Session 15's own Mongo compound index already enforced — same invariant, more structural mechanism. `CHECK (role IN (...))` is new: a real, database-level restriction on a column's own allowed values.

JOIN — the real payoff

A real query like `SELECT w.name, u.name, wm.role FROM workspace_members wm JOIN workspaces w ON ... JOIN users u ON ...` answers, in one statement, a question the real Mongo-backed app's own services currently answer with two separate queries and manual, in-code stitching. This is the concrete, structural reason data was deliberately split into separate tables in the first place.

Teamwork: FOREIGN KEY delete behavior (you discover this — it is not demonstrated live)

Every foreign key built live today has a real, silent default: Postgres refuses a delete that would leave a reference dangling, unless told otherwise. Teamwork researches the real alternatives (`ON DELETE CASCADE`, `ON DELETE SET NULL`) and classifies every real foreign key in the finished schema into one of three real categories — none of this is shown live.

Homework: genuinely new questions — rows with no match, and whole groups of rows

Every real query shown live or in teamwork looks at rows one at a time, and only where a match existed on both sides (even a JOIN, which just reconnects related single rows). Homework introduces `LEFT JOIN` — keeping a row even when it has no match at all, like a project with zero issues — combined with an aggregate function (`COUNT`) and `GROUP BY`, answering real questions about a whole GROUP of rows at once, like "how many issues does each project have." Nobody has shown you any of this live.

Common beginner mistakes

See `student/notes/STUDENT_NOTES-EN.md`.

A preview of what's still missing

This session builds four (live) plus two (teamwork) real tables, with every real constraint and delete behavior proven — but formal relational-design theory, indexes created beyond what PRIMARY KEY/UNIQUE already create, and transactions are all still completely open. A real activity-history table from the same frozen

domain model doesn't exist yet either. None of this is forgotten — it is deliberate, real future work for the next database session.

Not yet

Relational-design theory as named, formal rules; indexes created beyond what already exists; transactions; any object-relational mapping tool or schema-migration workflow; any frontend framework; an activity-history table.

Glossary

- **Table** — a fixed, named set of typed columns shared by every row.
- **Primary key** — the column (or columns) that uniquely identifies each row.
- **Foreign key** — a column that must match a real value already present in another table's own key column, enforced by the database itself.
- **Referential integrity** — the real guarantee that a foreign key always points at a row that genuinely exists.
- **Join table** — a table that exists purely to connect two other tables in a many-to-many relationship, optionally carrying its own attributes.
- **JOIN** — a query clause that reconnects rows across tables at query time, using a real foreign key relationship.
- **LEFT JOIN** — a JOIN that keeps every row from the left-hand table even when there is no matching row on the right, filling the missing columns with NULL instead of dropping the row.
- **Cascade** — a delete behavior that propagates automatically to dependent rows.
- **Aggregate function** — a function (like `COUNT`) that summarizes a group of rows into one value.

Review questions

1. In Mongo, a compound unique index stopped two `WorkspaceMember` rows for the same user in the same workspace. What stops it today, and is it a different mechanism or the same real idea?
2. Why does `workspace_members` need its own real table, rather than an array column on `workspaces`?
3. `issues.project_id` and `issues.assignee_id` are both real foreign keys built in the same session. Why does one use `CASCADE` and the other `SET NULL`?
4. What does Postgres actually do, by default, when no `ON DELETE` clause is named at all?
5. What would the real Mongo-backed app have to do, in code, to answer the same question a single real JOIN query answers here?

Answers

1. The exact same real invariant, now enforced by a real, database-level `UNIQUE` constraint on the pair of columns instead of a Mongo index — a stricter, more structural version of the same idea.
2. An array column has nowhere for a per-member attribute (`role`) to live, and the database could no longer enforce "one role per member" as a real structural constraint.

3. They represent different relationship kinds — an issue only exists because its project exists (CASCADE), but an issue's own history matters more than a stale assignee reference (SET NULL).
4. It refuses the delete outright if any row still references the target (`NO ACTION`, effectively `RESTRICT`) — not CASCADE.
5. Two separate queries, with the application doing the joining itself, in code — exactly what this app's own real Mongo-backed services already do today.

What you should be able to explain now

- The real difference between a document's own flexible shape and a table's own fixed, typed columns — and why that's a genuine tradeoff, never "SQL is better."
- What PRIMARY KEY, UNIQUE, FOREIGN KEY, and CHECK each actually enforce, structurally, at the database level.
- Why a many-to-many relationship with its own attribute needs a real join table.
- Why FOREIGN KEY delete behavior is a deliberate, per-relationship choice with at least three real, legitimate answers — never a single universal rule.